

Laporan Program: RSA-Based Secure File Encryption & Decryption



Mata Kuliah: Kriptografi

Disusun oleh:

Kanafira Vanesha Putri

5027241010

Fika Arka Nuriyah

5027241071

S. Farhan Baig

5027241097

Kelompok: Kelompok 11

Repositori: <https://github.com/pikaapik-io/RSA-Secure-File-Encryption-Decryption>

1. Pendahuluan

Pengamanan data digital merupakan kebutuhan mendasar dalam sistem informasi. Salah satu algoritma kriptografi kunci publik yang paling dikenal adalah RSA (Rivest-Shamir-Adleman), yang keamanannya bertumpu pada sulitnya memfaktorkan bilangan bulat berukuran besar menjadi faktor primanya. Berbeda dengan algoritma simetris seperti AES atau DES yang memakai satu kunci yang sama dan bergantung pada S-box, substitusi, serta permutasi, RSA memakai sepasang kunci (publik dan privat) dan murni berbasis aritmetika modular bilangan besar.

Laporan ini mendokumentasikan program enkripsi dan dekripsi file berbasis RSA yang seluruh tahapannya diimplementasikan secara manual tanpa library atau framework kriptografi, mulai dari pembangkitan bilangan prima, pembangkitan kunci, enkripsi, hingga dekripsi.

2. Tujuan

1. Memahami cara kerja algoritma RSA pada setiap tahapannya.
2. Mengimplementasikan pembangkitan kunci RSA (uji keprimaan, hitung n , $\phi(n)$, e , dan d) tanpa library kriptografi.
3. Mengimplementasikan enkripsi dan dekripsi pada file berformat apa pun (teks, gambar, PDF, dan lainnya).
4. Menguji kebenaran program dan menganalisis keamanan serta keterbatasannya.

3. Landasan Teori

3.1 Algoritma RSA

RSA bekerja pada aritmetika modular dengan rumus berikut.

Tahap	Rumus
Modulus	$n = p \times q$
Fungsi totient	$\phi(n) = (p - 1)(q - 1)$
Kunci publik	e dengan $1 < e < \phi(n)$ dan $\gcd(e, \phi(n)) = 1$
Kunci privat	$d \equiv e^{-1} \pmod{\phi(n)}$, yaitu $(e \times d) \bmod \phi(n) = 1$
Enkripsi	$c = m^e \bmod n$
Dekripsi	$m = c^d \bmod n$

Kunci publik adalah pasangan (n, e) dan kunci privat adalah pasangan (n, d) .

3.2 Kebenaran Dekripsi

Karena $e \cdot d = 1 + k \cdot \phi(n)$ untuk suatu bilangan bulat k , maka berdasarkan teorema Euler:

$$c^d = (m^e)^d = m^{(e \cdot d)} = m^{(1 + k \cdot \phi(n))} = m \times (m^{\phi(n)})^k \equiv m \times 1^k \equiv m \pmod{n}$$

3.3 Uji Keprimaan Miller-Rabin

Untuk menemukan bilangan prima berukuran ratusan digit digunakan uji probabilitas Miller-Rabin. Bilangan $x - 1$ ditulis sebagai $2^s \times t$ dengan t ganjil, lalu diuji dengan basis acak a . Bilangan dinyatakan komposit bila $a^t \bmod x$ tidak bernilai 1 atau $x - 1$ dan tidak ada kuadrat berikutnya yang bernilai $x - 1$. Program memakai 40 putaran sehingga peluang kesalahan sangat kecil.

3.4 Algoritma Euclid yang Diperluas

Kunci privat d dihitung dengan Extended Euclid, yaitu mencari x dan y sehingga $a \cdot x + b \cdot y = \gcd(a, b)$. Untuk $a = e$ dan $b = \phi(n)$ dengan $\gcd = 1$, nilai $x \bmod \phi(n)$ adalah invers modular d .

3.5 Pangkat Modular (Square-and-Multiply)

Menghitung $m^e \bmod n$ secara langsung tidak praktis karena hasil antaranya sangat besar. Metode square-and-multiply memotong hasil dengan $\bmod n$ pada setiap langkah sehingga hanya memerlukan $O(\log e)$ perkalian modular.

3.6 Padding

RSA tanpa padding bersifat deterministik: plaintext yang sama selalu menghasilkan ciphertext yang sama, dan pesan pendek rentan ditebak. Program ini memakai padding acak bergaya PKCS#1 v1.5 tipe 2:

$EM = 0x00 \parallel 0x02 \parallel PS \parallel 0x00 \parallel DATA$

dengan PS berupa minimal 8 byte acak yang tidak bernilai nol. Dengan padding ini, enkripsi file yang sama menghasilkan ciphertext yang berbeda setiap kali, dan saat dekripsi dapat dideteksi apakah kunci yang dipakai benar.

4. Perancangan

4.1 Alur Pembangkitan Kunci

FUNCTION GenerateKeys(bits):

 REPEAT

$p \leftarrow \text{GeneratePrime}(\text{bits} / 2)$

$q \leftarrow \text{GeneratePrime}(\text{bits} / 2)$

 UNTIL $p \neq q$

$n \leftarrow p \times q$

$\phi \leftarrow (p - 1) \times (q - 1)$

$e \leftarrow 65537$

 IF $\gcd(e, \phi) \neq 1$ THEN ulangi dari awal

```

d ← ModInverse(e, phi)
RETURN publicKey = (n, e), privateKey = (n, d)

```

4.2 Alur Enkripsi File

```

FUNCTION EncryptFile(file, (n, e)):
  k ← jumlah byte dari n
  maxData ← k - 11
  data ← baca semua byte file
  FOR setiap potongan chunk (≤ maxData byte) dari data:
    EM ← 00 || 02 || [byte acak ≠ 0] || 00 || chunk
    m ← ByteKeInteger(EM)
    c ← ModPow(m, e, n)
    tulis IntegerKeByte(c, k byte) ke file .enc

```

4.3 Alur Dekripsi File

```

FUNCTION DecryptFile(file.enc, (n, d)):
  k ← jumlah byte dari n
  FOR setiap blok k byte dari file.enc:
    c ← ByteKeInteger(blok)
    m ← ModPow(c, d, n)
    EM ← IntegerKeByte(m, k byte)
    validasi EM diawali 00 02, cari byte 00 pemisah
    chunk ← bagian setelah byte 00 pemisah
    tulis chunk ke file hasil

```

4.4 Contoh Perhitungan Angka Kecil

Contoh ini hanya untuk memahami rumus (program sebenarnya memakai bilangan jauh lebih besar).

Langkah	Perhitungan
Pilih prima	$p = 61, q = 53$
Modulus	$n = 61 \times 53 = 3233$
Totient	$\varphi(n) = 60 \times 52 = 3120$
Kunci publik	$e = 17$, karena $\gcd(17, 3120) = 1$
Kunci privat	$d = 17^{-1} \bmod 3120 = 2753$ ($17 \times 2753 = 46801 = 15 \times 3120 + 1$)

Langkah	Perhitungan
Enkripsi $m = 65$	$c = 65^{17} \bmod 3233 = 2790$
Dekripsi	$m = 2790^{2753} \bmod 3233 = 65$

5. Implementasi

Program ditulis dalam bahasa Python 3 pada berkas `rsa_file_crypto.py`. Tidak ada library kriptografi yang digunakan. Modul bawaan yang dipakai hanya `os` (untuk `os.urandom` sebagai sumber bilangan acak) dan `sys`. Tipe `int` Python mendukung bilangan bulat berukuran tak terbatas sehingga cocok untuk aritmetika RSA.

5.1 Struktur Fungsi

Tahap	Fungsi	Keterangan
Utilitas	<code>random_bits</code> , <code>random_range</code>	Bilangan acak dari <code>os.urandom</code>
Matematika	<code>mod_pow</code> , <code>gcd</code> , <code>ext_gcd</code> , <code>mod_inverse</code>	Square-and-multiply, Euclid, invers modular
Prima	<code>is_prime</code> , <code>generate_prime</code>	Miller-Rabin 40 putaran
Kunci	<code>generate_keys</code> , <code>save_key</code> , <code>load_key</code>	Membuat dan menyimpan kunci
Padding	<code>pad_block</code> , <code>unpad_block</code>	Padding acak bergaya PKCS#1 v1.5
Enkripsi dan dekripsi	<code>encrypt_file</code> , <code>decrypt_file</code>	Pemrosesan file per blok
Antarmuka	<code>main</code>	Menu berbasis teks

5.2 Potongan Kode Inti

Pangkat modular (square-and-multiply):

```
def mod_pow(base, exp, mod):
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1:
            result = (result * base) % mod
        exp >>= 1
```

```

    base = (base * base) % mod
    return result

```

Invers modular dengan Extended Euclid:

```

def ext_gcd(a, b):
    x0, x1, y0, y1 = 1, 0, 0, 1
    while b:
        q = a // b
        a, b = b, a - q * b
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
    return a, x0, y0

```

```

def mod_inverse(e, phi):
    g, x, _ = ext_gcd(e, phi)
    if g != 1:
        raise ValueError("e dan phi tidak koprima")
    return x % phi

```

Enkripsi satu blok:

```

em = pad_block(chunk, k)      # padding
m = int.from_bytes(em, "big") # bytes -> integer
c = mod_pow(m, e, n)          # c = m^e mod n
out += c.to_bytes(k, "big")   # integer -> bytes

```

Dekripsi satu blok:

```

c = int.from_bytes(cipher[i*k:(i+1)*k], "big")
m = mod_pow(c, d, n)          # m = c^d mod n
em = m.to_bytes(k, "big")
out += unpad_block(em)        # hapus padding

```

5.3 Parameter dan Format Berkas

Parameter	Nilai
Eksponen publik e	65537
Ukuran kunci	Minimal 512 bit, default 1024 bit
Panjang blok cipher (k)	Jumlah byte dari n (128 byte untuk kunci 1024 bit)
Kapasitas data per blok	k – 11 byte

Parameter	Nilai
Putaran Miller-Rabin	40

Berkas kunci berupa teks dua baris berisi bilangan desimal.

- `public.key`: baris 1 adalah `n`, baris 2 adalah `e`.
- `private.key`: baris 1 adalah `n`, baris 2 adalah `d`.

Berkas `.enc` berisi rangkaian blok cipher berukuran `k` byte tanpa header tambahan.

5.4 Cara Menjalankan

`py rsa_file_crypto.py`

Menu	Fungsi
1	Bangkitkan kunci (menghasilkan <code>public.key</code> dan <code>private.key</code>) dan menampilkan langkah <code>p</code> , <code>q</code> , <code>n</code> , <code>φ</code> , <code>e</code> , <code>d</code>
2	Enkripsi file memakai <code>public.key</code>
3	Dekripsi file memakai <code>private.key</code>
0	Keluar

6. Pengujian dan Hasil

Pengujian dilakukan dengan membandingkan isi file sebelum enkripsi dan sesudah dekripsi.

No	Skenario	Hasil yang diharapkan	Hasil
1	Pembangkitan kunci 1024 bit	Kunci terbentuk dan $(e \times d) \bmod \phi = 1$	Berhasil (sekitar 0,13 detik)
2	Enkripsi lalu dekripsi file acak 5 KB	File hasil identik dengan file asli	Berhasil (identik)
3	Enkripsi lalu dekripsi file kosong	File hasil kosong	Berhasil
4	Dekripsi dengan kunci privat yang salah	Ditolak dengan pesan kesalahan	Ditolak: "Padding tidak valid (kunci salah atau file rusak)"

No	Skenario	Hasil yang diharapkan	Hasil
5	Enkripsi file teks tes.txt	File .enc berisi byte acak tidak terbaca	Sesuai

Penyimpanan tangkapan layar nomor 1, 2, 3 dan hasil perbandingan file aslinya

Untuk memverifikasi hasil di Windows PowerShell:

```
fc.exe tes.txt tes_hasil.txt
```

Keluaran "No differences encountered" menunjukkan file hasil dekripsi identik dengan file asli.

7. Analisis

7.1 Keamanan

- Keamanan RSA bertumpu pada sulitnya memfaktorkan n menjadi $p \times q$. Tanpa p dan q , $\phi(n)$ tidak dapat dihitung sehingga d tidak dapat diturunkan dari (n, e) .
- Padding acak membuat enkripsi bersifat probabilistik: file yang sama menghasilkan ciphertext berbeda pada setiap enkripsi.
- Sumber acak memakai `os.urandom` yang berbasis generator acak sistem operasi.
- Kunci privat harus dijaga. Karena itu `private.key` dimasukkan ke `.gitignore` agar tidak terunggah ke GitHub.

7.2 Keterbatasan

- Kinerja.** RSA memakai operasi bilangan besar sehingga jauh lebih lambat daripada cipher simetris. Program cocok untuk file berukuran kecil sampai beberapa MB. Pada praktik nyata, RSA dipakai untuk mengenkripsi kunci AES dan AES yang mengenkripsi file (skema hybrid).
- Padding.** PKCS#1 v1.5 diketahui rentan terhadap serangan padding oracle bila diterapkan pada layanan yang membocorkan hasil validasi padding. Standar modern menganjurkan OAEP.
- Tidak ada autentikasi.** Program tidak membuat tanda tangan digital atau hash untuk mendeteksi perubahan file.
- Implementasi buatan sendiri.** Tidak tahan terhadap serangan side-channel seperti timing attack. Untuk lingkungan produksi, gunakan library kriptografi yang telah teruji.
- Ukuran kunci.** Kunci 1024 bit cukup untuk demonstrasi, tetapi keamanan nyata memerlukan minimal 2048 bit.

7.3 Pengembangan Lanjutan

- Optimasi dekripsi dengan Chinese Remainder Theorem (CRT).
- Penggantian padding dengan OAEP.

7.4 Pengembangan Antarmuka Web

Untuk meningkatkan kemudahan penggunaan, program dikembangkan dengan antarmuka web berbasis Flask. Pengembangan ini tidak mengganti implementasi algoritma RSA yang telah dibahas pada bagian sebelumnya. Fungsi pembangkitan kunci, padding, enkripsi, dan dekripsi tetap berada pada `rsa_file_crypto.py` dan digunakan kembali oleh backend web.

Backend web terdapat pada `app.py`. Berkas ini menerima masukan dari browser, memvalidasi berkas atau teks, memanggil fungsi RSA dari `rsa_file_crypto.py`, menyimpan hasil operasi secara terkontrol, dan menyediakan hasil untuk diunduh. Dengan demikian, versi web dan versi terminal menggunakan inti kriptografi yang sama.

7.4.1 Fitur Antarmuka Web

Antarmuka web menyediakan pembangkitan pasangan public key dan private key, enkripsi berkas, dekripsi berkas, serta dua jenis masukan. Pengguna dapat mengunggah berkas melalui file picker atau menempelkan teks secara langsung. Pada mode teks, ciphertext ditampilkan dalam bentuk Base64 sehingga dapat disalin dan ditempelkan kembali ketika melakukan dekripsi.

Public key dan private key dapat digunakan melalui unggahan berkas atau melalui input teks. Format input teks terdiri atas dua bilangan desimal, yaitu n dan e untuk public key, serta n dan d untuk private key. Format ini tetap kompatibel dengan format kunci pada versi terminal.

7.4.2 Pemisahan Komponen Program

Komponen	Fungsi
<code>rsa_file_crypto.py</code>	Implementasi manual RSA dan antarmuka menu terminal.
<code>app.py</code>	Backend Flask yang memanggil fungsi RSA dari <code>rsa_file_crypto.py</code> .
<code>templates/</code> dan <code>static/</code>	HTML, CSS, serta JavaScript antarmuka web.
<code>tests/test_app.py</code>	Pengujian integrasi untuk alur web.

7.4.3 Penggunaan Dua Versi

Versi web dapat dijalankan dengan beberapa cara. Cara yang disarankan adalah menjalankan `./run.sh` dari root repository karena script tersebut membuat virtual environment dan memasang dependency secara otomatis. Setelah dependency tersedia, server web juga dapat dijalankan secara langsung dengan `python app.py` atau `python3 app.py`. Aplikasi kemudian diakses melalui `http://127.0.0.1:5000`.

Versi terminal dijalankan secara terpisah dengan perintah `python rsa_file_crypto.py` atau `python3 rsa_file_crypto.py`. Perintah tersebut menampilkan menu interaktif pada terminal untuk membangkitkan kunci, mengenkripsi file, dan mendekripsi file. Kedua versi menggunakan pasangan kunci dan fungsi kriptografi yang sama, sehingga penambahan antarmuka web tidak menghilangkan program terminal.

Gambar berikut memperlihatkan halaman utama aplikasi web dan penggunaan versi terminal.

Gambar 1. Antarmuka Web RSA File Encryption

CRYPTOGRAPHY WORKSPACE

RSA File Encryption

About RSA

A simple workspace for RSA-based file encryption and decryption.

FILE WORKSPACE

Uploads up to 16 MB

Protect a file

Encrypt file Decrypt file

Upload a file

Paste text

File to encrypt

Choose a file

The selected file will stay in the app's private workspace.

Key

Choose File

Output filename

Optional

Encrypt file

KEY MANAGEMENT

Generate a pair

Create a public/private pair for this browser session.
Keep the private key somewhere safe.

Key size

1024 bit

Generate RSA key pair

Academic demonstration only.
Not intended for production secrets.

HOW IT WORKS

Three steps: generate keys, encrypt with the public key, decrypt with the private key.

Gambar 2. Penggunaan Versi Terminal RSA File Encryption

```
Terminal - rsa_file_crypto.py

$ python3 rsa_file_crypto.py

===== RSA FILE ENCRYPTION / DECRYPTION =====
1. Bangkitkan kunci (public.key & private.key)
2. Enkripsi file
3. Dekripsi file
0. Keluar
Pilih: 1

Ukuran kunci (bit, min 512) [1024]: 512
[1] Membangkitkan prima p (256 bit) ...
    p ditemukan setelah 18 kandidat
[2] Membangkitkan prima q (256 bit) ...
    q ditemukan setelah 9 kandidat
[3] n = p * q                -> 512 bit
[5] e = 65537 (gcd(e, phi) = 1 terpenuhi)
[6] d = e^-1 mod phi        -> 511 bit
[7] Verifikasi (e*d) mod phi = 1 (harus 1)

Kunci tersimpan: public.key dan private.key
JAGA private.key, jangan dibagikan!

Pilih: 2
File yang dienkrpsi: contoh_input.txt
File hasil [contoh_input.txt.enc]: contoh_input.txt.enc
File kunci publik [public.key]: public.key
Enkripsi selesai -> contoh_input.txt.enc
```

9

- Skema hybrid RSA dengan cipher simetris untuk file berukuran besar.
- Penambahan tanda tangan digital dan verifikasi hash.

8. Kesimpulan

Program enkripsi dan dekripsi file berbasis RSA telah berhasil diimplementasikan tanpa library kriptografi. Seluruh tahapan, yaitu pembangkitan bilangan prima dengan Miller-Rabin, perhitungan n , $\phi(n)$, e , dan d dengan Extended Euclid, enkripsi $c = m^e \bmod n$, dan dekripsi $m = c^d \bmod n$, dikerjakan secara manual dan dapat ditelusuri langkah demi langkah. Pengujian menunjukkan file hasil dekripsi identik dengan file asli, dan dekripsi dengan kunci yang salah ditolak. Program ini ditujukan untuk pembelajaran. Untuk kebutuhan produksi, diperlukan padding OAEP, ukuran kunci yang lebih besar, skema hybrid, dan library kriptografi yang telah teruji.

Daftar Pustaka

1. Rivest, R. L., Shamir, A., dan Adleman, L. (1978). *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*. Communications of the ACM, 21(2), 120-126.
2. Stallings, W. *Cryptography and Network Security: Principles and Practice*. Pearson.
3. Menezes, A., van Oorschot, P., dan Vanstone, S. *Handbook of Applied Cryptography*. CRC Press.
4. RFC 8017: *PKCS #1: RSA Cryptography Specifications Version 2.2*. IETF.